



SDK

Cliente JS con seguridad de tipos para el servidor opencode.

El SDK opencode JS/TS proporciona un cliente con seguridad de tipos para interactuar con el servidor. Úselo para crear integraciones y controlar opencode mediante programación.

Más información sobre cómo funciona el servidor. Para ver ejemplos, consulte los proyectos creados por la comunidad.

Instalar

Instale el SDK desde npm:

```
npm install @opencode-ai/sdk
```

Crear cliente

Cree una instancia de opencode:

```
import { createOpencode } from "@opencode-ai/sdk"

const { client } = await createOpencode()
```

Esto inicia tanto un servidor como un cliente.

Opciones

OPCIÓN	TIPO	DESCRIPCIÓN	PREDETERMINADO
<code>`hostname`</code>	<code>`string`</code>	Nombre de host del servidor	<code>`127.0.0.1`</code>
<code>`port`</code>	<code>`number`</code>	Puerto del servidor	<code>`4096`</code>
<code>`signal`</code>	<code>`AbortSignal`</code>	Señal de aborto para cancelación	<code>`undefined`</code>
<code>`timeout`</code>	<code>`number`</code>	Tiempo de espera en ms para inicio del servidor	<code>`5000`</code>
<code>`config`</code>	<code>`Config`</code>	Objeto de configuración	<code>`{}`</code>

Configuración

Puede pasar un objeto de configuración para personalizar el comportamiento. La instancia aún recoge su ``opencode.json``, pero puede anular o agregar configuración en línea:

```
import { createOpencode } from "@opencode-ai/sdk"

const opencode = await createOpencode({
  hostname: "127.0.0.1",
  port: 4096,
  config: {
    model: "anthropic/claude-3-5-sonnet-20241022",
  },
})

console.log(`Server running at ${opencode.server.url}`)

opencode.server.close()
```

Solo cliente

Si ya tiene una instancia en ejecución de opencode, puede crear una instancia de cliente para conectarse a ella:

```
import { createOpencodeClient } from "@opencode-ai/sdk"

const client = createOpencodeClient({
  baseUrl: "http://localhost:4096",
})
```

Opciones

OPCIÓN	TIPO	DESCRIPCIÓN	PREDETERMINADO
<code>`baseUrl`</code>	<code>`string`</code>	URL del servidor	<code>`http://localhost:4096`</code>
<code>`fetch`</code>	<code>`function`</code>	Implementación de recuperación personalizada	<code>`globalThis.fetch`</code>
<code>`parseAs`</code>	<code>`string`</code>	Método de análisis de respuesta	<code>`auto`</code>
<code>`responseStyle`</code>	<code>`string`</code>	Estilo de devolución: <code>`data`</code> o <code>`fields`</code>	<code>`fields`</code>
<code>`throwOnError`</code>	<code>`boolean`</code>	Lanzar errores en lugar de devolver	<code>`false`</code>

Tipos

El SDK incluye definiciones TypeScript para todos los tipos API. Importarlos directamente:

```
import type { Session, Message, Part } from "@opencode-ai/sdk"
```

Todos los tipos se generan a partir de la especificación OpenAPI del servidor y están disponibles en el archivo de tipos.

Errores

El SDK puede generar errores que puedes detectar y manejar:

```
try {  
  await client.session.get({ path: { id: "invalid-id" } })  
} catch (error) {  
  console.error("Failed to get session:", (error as Error).message)  
}
```

Salida Estructurada

Puede solicitar una salida JSON estructurada del modelo especificando un `format` con un esquema JSON. El modelo utilizará una herramienta `StructuredOutput` para devolver un JSON validado que coincida con su esquema.

Uso Básico

```
const result = await client.session.prompt({
  path: { id: sessionId },
  body: {
    parts: [{ type: "text", text: "Research Anthropic and provide company
info" }],
    format: {
      type: "json_schema",
      schema: {
        type: "object",
        properties: {
          company: { type: "string", description: "Company name" },
          founded: { type: "number", description: "Year founded" },
          products: {
            type: "array",
            items: { type: "string" },
            description: "Main products",
          },
        },
        required: ["company", "founded"],
      },
    },
  },
})

// Access the structured output
console.log(result.data.info.structured_output)
// { company: "Anthropic", founded: 2021, products: ["Claude", "Claude API"] }
```

Tipos de Formato de Salida

TIPO	DESCRIPCIÓN
`text`	Predeterminado. Respuesta de texto estándar (sin salida estructurada)
`json_schema`	Devuelve JSON validado que coincide con el esquema proporcionado

Formato de Esquema JSON

Cuando use ``type: 'json_schema'``, proporcione:

CAMPO	TIPO	DESCRIPCIÓN
<code>`type`</code>	<code>`'json_schema'`</code>	Requerido. Especifica el modo de esquema JSON
<code>`schema`</code>	<code>`object`</code>	Requerido. Objeto JSON Schema que define la estructura de salida
<code>`retryCount`</code>	<code>`number`</code>	Opcional. Número de reintentos de validación (predeterminado: 2)

Manejo de Errores

Si el modelo no logra producir una salida estructurada válida después de todos los reintentos, la respuesta incluirá un ``StructuredOutputError``:

```
if (result.data.info.error?.name === "StructuredOutputError") {
  console.error("Failed to produce structured output:",
result.data.info.error.message)
  console.error("Attempts:", result.data.info.error.retries)
}
```

Mejores Prácticas

Proporcione descripciones claras en las propiedades de su esquema para ayudar al modelo a entender qué datos extraer

Use ``required`` para especificar qué campos deben estar presentes

Mantenga los esquemas enfocados - los esquemas anidados complejos pueden ser más difíciles de completar correctamente para el modelo

Establezca un ``retryCount`` apropiado - aumente para esquemas complejos, disminuya para simples

API

El SDK expone todas las API del servidor a través de un cliente con seguridad de tipos.

Global

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`global.healthh()`</code>	Verificar el estado y la versión del servidor	<code>`{ healthy: true, version: string }`</code>

Ejemplos

```
const health = await client.global.health()
console.log(health.data.version)
```

Aplicación

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`app.log()`</code>	Escribe una entrada de registro	<code>`boolean`</code>
<code>`app.agents()`</code>	Listar todos los agentes disponibles	<code><u>`Agent[]`</u></code>

Ejemplos

```
// Write a log entry
await client.app.log({
  body: {
    service: "my-app",
    level: "info",
    message: "Operation completed",
  },
})

// List available agents
const agents = await client.app.agents()
```

Proyecto

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`project.list()`</code>	Listar todos los proyectos	<code><u>`Project[]`</u></code>
<code>`project.current()`</code>	Obtener proyecto actual	<code><u>`Project`</u></code>

Ejemplos

```
// List all projects
const projects = await client.project.list()

// Get current project
const currentProject = await client.project.current()
```


Ruta

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`path.get()`</code>	Obtener ruta actual	<code><u>`Path`</u></code>

Ejemplos

```
// Get current path information
const pathInfo = await client.path.get()
```

Configuración

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`config.get()`</code>	Obtener información de configuración	<code><u>`Config`</u></code>
<code>`config.providers()`</code>	Lista de proveedores y modelos predeterminados	<code>`{ providers: ``Provider[]``, default: { [key: string]: string } }`</code>

Ejemplos

```
const config = await client.config.get()

const { providers, default: defaults } = await client.config.providers()
```

Sesiones

MÉTODO	DESCRIPCIÓN	NOTAS
<code>`session.list()`</code>	Listar sesiones	Devuelve <u><code>`Session[]`</code></u>
<code>`session.get({ path })`</code>	Obtener sesión	Devuelve <u><code>`Session`</code></u>
<code>`session.children({ path })`</code>	Listar sesiones secundarias	Devuelve <u><code>`Session[]`</code></u>
<code>`session.create({ body })`</code>	Crear sesión	Devuelve <u><code>`Session`</code></u>
<code>`session.delete({ path })`</code>	Eliminar sesión	Devuelve <code>`boolean`</code>
<code>`session.update({ path, body })`</code>	Actualizar propiedades de sesión	Devuelve <u><code>`Session`</code></u>
<code>`session.init({ path, body })`</code>	Analizar aplicación y crear <code>`AGENTS.md`</code>	Devuelve <code>`boolean`</code>
<code>`session.abort({ path })`</code>	Cancelar una sesión en ejecución	Devuelve <code>`boolean`</code>
<code>`session.share({ path })`</code>	Compartir sesión	Devuelve <u><code>`Session`</code></u>
<code>`session.unshare({ path })`</code>	Dejar de compartir sesión	Devuelve <u><code>`Session`</code></u>
<code>`session.summarize({ path, body })`</code>	Resumir sesión	Devuelve <code>`boolean`</code>

MÉTODO	DESCRIPCIÓN	NOTAS
<code>`session.messages({ path })`</code>	Listar mensajes en una sesión	Devuelve <code>{ info: `Message`, parts: `Part[]` }[]`</code>
<code>`session.message({ path })`</code>	Obtener detalles del mensaje	Devuelve <code>{ info: `Message`, parts: `Part[]` }</code>
<code>`session.prompt({ path, body })`</code>	Enviar mensaje rápido	<code>body.noReply: true`</code> devuelve <code>UserMessage</code> (solo contexto). El valor predeterminado devuelve <code>AssistantMessage`</code> con respuesta de IA. Admite <code>body.outputFormat`</code> para <u>salida estructurada</u>
<code>`session.command({ path, body })`</code>	Enviar comando a la sesión	Devuelve <code>{ info: `AssistantMessage`, parts: `Part[]` }</code>
<code>`session.shell({ path, body })`</code>	Ejecute un comando de shell	Devuelve <code>AssistantMessage`</code>
<code>`session.revert({ path, body })`</code>	Revertir un mensaje	Devuelve <code>Session`</code>
<code>`session.unrevert({ path })`</code>	Restaurar mensajes revertidos	Devuelve <code>Session`</code>
<code>`postSessionByIdPermissionsByPermissionId({ path, body })`</code>	Responder a una solicitud de permiso	Devuelve <code>boolean`</code>

Ejemplos

```
// Create and manage sessions
const session = await client.session.create({
  body: { title: "My session" },
})

const sessions = await client.session.list()

// Send a prompt message
const result = await client.session.prompt({
  path: { id: session.id },
  body: {
    model: { providerID: "anthropic", modelID: "claude-3-5-sonnet-20241022" },
    parts: [{ type: "text", text: "Hello!" }],
  },
})

// Inject context without triggering AI response (useful for plugins)
await client.session.prompt({
  path: { id: session.id },
  body: {
    noReply: true,
    parts: [{ type: "text", text: "You are a helpful assistant." }],
  },
})
```

Archivos

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`find.text({ query })`</code>	Buscar texto en archivos	Matriz de objetos coincidentes con <code>`path`</code> , <code>`lines`</code> , <code>`line_number`</code> , <code>`absolute_offset`</code> , <code>`submatches`</code>

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`find.files({ query })`</code>	Buscar archivos y directorios por nombre	<code>`string[]`</code> (rutas)
<code>`find.symbols({ query })`</code>	Buscar símbolos del espacio de trabajo	<code><u>`Symbol[]`</u></code>
<code>`file.read({ query })`</code>	Leer un archivo	<code>`{ type: "raw"   "patch", content: string }`</code>
<code>`file.status({ query? })`</code>	Obtener el estado de los archivos rastreados	<code><u>`File[]`</u></code>

``find.files`` admite algunos campos de consulta opcionales:

``type``: ``"file"`` o ``"directory"``

``directory``: anula la raíz del proyecto para la búsqueda.

``limit``: resultados máximos (1-200)

Ejemplos

```
// Search and read files
const textResults = await client.find.text({
  query: { pattern: "function.*opencode" },
})

const files = await client.find.files({
  query: { query: "*.ts", type: "file" },
})

const directories = await client.find.files({
  query: { query: "packages", type: "directory", limit: 20 },
})

const content = await client.file.read({
  query: { path: "src/index.ts" },
})
```

TUI

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`tui.appendPrompt({ body })`</code>	Agregar texto al mensaje	<code>`boolean`</code>
<code>`tui.openHelp()`</code>	Abra el cuadro de diálogo de ayuda	<code>`boolean`</code>
<code>`tui.openSessions()`</code>	Abrir el selector de sesiones	<code>`boolean`</code>
<code>`tui.openThemes()`</code>	Abra el selector de temas	<code>`boolean`</code>
<code>`tui.openModels()`</code>	Abrir el selector de modelo	<code>`boolean`</code>
<code>`tui.submitPrompt()`</code>	Enviar el mensaje actual	<code>`boolean`</code>
<code>`tui.clearPrompt()`</code>	Borrar el mensaje	<code>`boolean`</code>
<code>`tui.executeCommand({ body })`</code>	Ejecutar un comando	<code>`boolean`</code>

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`tui.showToast({ body })`</code>	Mostrar notificación del brindis	<code>`boolean`</code>

Ejemplos

```
// Control TUI interface
await client.tui.appendPrompt({
  body: { text: "Add this to prompt" },
})

await client.tui.showToast({
  body: { message: "Task completed", variant: "success" },
})
```

Autenticación

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`auth.set({ ... })`</code>	Establecer credenciales de autenticación	<code>`boolean`</code>

Ejemplos

```
await client.auth.set({
  path: { id: "anthropic" },
  body: { type: "api", key: "your-api-key" },
})
```

Eventos

MÉTODO	DESCRIPCIÓN	RESPUESTA
<code>`event.subscribe()`</code>	Transmisión de eventos enviados por el servidor	Transmisión de eventos enviados por el servidor

Ejemplos

```
// Listen to real-time events
const events = await client.event.subscribe()
for await (const event of events.stream) {
  console.log("Event:", event.type, event.properties)
}
```

-  [Edita esta página](#)
-  [Found a bug? Open an issue](#)
-  [Join our Discord community](#)

© Anomaly

Última actualización: 21 may 2026

🌐 Español 